```
from google.colab import drive
drive.mount('/content/drive/')
```

```
Mounted at /content/drive/
```

```
!pip install spectral
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting spectral
  Downloading spectral-0.23.1-py3-none-any.whl (212 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 212.9/212.9 KB 18.6 MB/s eta 0:00:00
Requirement already satisfied: numpy in /usr/local/lib/python3.9/dist-packages (from spectral) (1.22.4)
Installing collected packages: spectral
Successfully installed spectral-0.23.1
```

```
%tensorflow_version 2.x
import tensorflow as tf
import random
from tensorflow.keras import Sequential,layers
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint
import statistics
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, cohen_kappa_score
import tensorflow_probability as tfp
from operator import truediv
from tensorflow.compat.v1.distributions import Bernoulli
from plotly.offline import init_notebook_mode

import numpy as np
```

Saved successfully!

```
import spectral
import tensorflow as tf
from tensorflow.python.keras import backend as K

init_notebook_mode(connected=True)
%matplotlib inline
```

```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
```

```
# Global variables
im_width, im_height, im_depth, im_channel = 11,11,30, 1
mc_loss_weight = 5
```

Model Construction

```
def calc_euclidian_dists(x, y):
  # x : (n,d)
  # y : (m,d)
    n = x.shape[0]
    m = y.shape[0]
    x = tf.tile(tf.expand_dims(x, 1), [1, m, 1])
    y = tf.tile(tf.expand_dims(y, 0), [n, 1, 1])
    return tf.reduce_mean(tf.math.pow(x - y, 2), 2)
```

```
def _bernoulli(shape, mean):
    return tf.nn.relu(tf.sign(mean - tf.random.uniform(shape, minval=0, maxval=1, dtype=tf.float32)))
```

```
class DropBlock3D(tf.keras.layers.Layer):
    def __init__(self, keep_prob, block_size, scale=True, **kwargs):
        super(DropBlock3D, self).__init__(**kwargs)
        self.keep_prob = float(keep_prob) if isinstance(keep_prob, int) else keep_prob
        self.block_size = int(block_size)
        self.scale = tf.constant(scale, dtype=tf.bool) if isinstance(scale, bool) else scale

    def compute_output_shape(self, input_shape):
        return input_shape

    def build(self, input_shape):
        assert len(input_shape) == 5
        _, self.d, self.h, self.w, self.channel = input_shape.as_list()
```

```
        # pad the mask
        p0 = (self.block_size - 1) // 2
        p1 = (self.block_size - 1) - p0
        self.padding = [[0, 0], [p0, p1], [p0, p1], [p0, p1], [0, 0]]
        self.set_keep_prob()
        super(DropBlock3D, self).build(input_shape)

    def call(self, inputs, training=True, **kwargs):
        def drop():
            mask = self._create_mask(tf.shape(inputs))
            output = inputs * mask
            output = tf.cond(self.scale,
                             true_fn=lambda: output * tf.compat.v1.to_float(tf.size(mask)) / tf.reduce_sum(mask),
                             false_fn=lambda: output)
            return output

        if training is None:
            training = K.learning_phase()
        output = tf.cond(tf.logical_or(tf.logical_not(training), tf.equal(self.keep_prob, 1.0)),
                         true_fn=lambda: inputs,
                         false_fn=drop)
        return output

    def set_keep_prob(self, keep_prob=None):
        """This method only supports Eager Execution"""
        if keep_prob is not None:
            self.keep_prob = keep_prob
        d, w, h = tf.compat.v1.to_float(self.d), tf.compat.v1.to_float(self.w), tf.compat.v1.to_float(self.h)
        self.gamma = ((1. - self.keep_prob) * (d * w * h) / (self.block_size ** 3) /
                      ((d - self.block_size + 1) * (w - self.block_size + 1) * (h - self.block_size + 1)))

    def _create_mask(self, input_shape):
        sampling_mask_shape = tf.stack([input_shape[0],
                                        self.d - self.block_size + 1,
                                        self.h - self.block_size + 1,
                                        self.w - self.block_size + 1,
                                        self.channel])
        mask = _bernoulli(sampling_mask_shape, self.gamma)
        mask = tf.pad(mask, self.padding)
        mask = tf.nn.max_pool3d(mask, [1, self.block_size, self.block_size, self.block_size, 1], [1, 1, 1, 1, 1], 'SAME')
        mask = 1 - mask
        return mask
```

Saved successfully!

```
# input_layer = layers.Input(shape = (im_height, im_width, im_depth, im_channel))
# out1 = layers.Conv3D(filters=8, kernel_size=(3,3,7), activation='relu',input_shape=(im_height, im_width, im_depth, im_channel),padding=
# out1 = DropBlock3D(0.7,3)(out1)
# out2 = layers.Conv3D(filters=16, kernel_size=(3,3,5), activation='relu',padding='same')(out1)
# out2 = DropBlock3D(0.7,3)(out2)
# out3 = layers.Conv3D(filters=32, kernel_size=(3,3,3), activation= 'relu')(out2)
# out3 = layers.Reshape((out3.shape[1], out3.shape[2], out3.shape[3]*out3.shape[4]))(out3)
# out3 = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(out3)
# out4 = layers.Flatten()(out3)
# out4 = layers.Dropout(0.4)(out4, training=True)
# out4 = layers.Dense(256, activation='relu')(out4)
# out5 = layers.Dropout(0.4)(out4,training=True)
# out5 = layers.Dense(128, activation='relu')(out5)
# model = Model(inputs=input_layer,outputs=out5)
# model.summary()


# Mayank's Architecture 1

# input_layer = layers.Input(shape = (im_height, im_width, im_depth, im_channel))
# out1 = layers.Conv3D(filters=8, kernel_size=(3,3,7), activation='relu',input_shape=(im_height, im_width, im_depth, im_channel),padding=
# out1 = DropBlock3D(0.7,3)(out1)
# out2 = layers.Conv3D(filters=16, kernel_size=(3,3,5), activation='relu',padding='same')(out1)
# out2 = DropBlock3D(0.7,3)(out2)
# out3_conv = layers.Conv3D(filters=32, kernel_size=(3,3,3), activation= 'relu', padding = 'same')(out2)
# out3 = DropBlock3D(0.7,3)(out3_conv)
# attention = layers.Attention()([out3, out3_conv])
# out3 = layers.Reshape((out3.shape[1], out3.shape[2], out3.shape[3]*out3.shape[4]))(attention)
# out3 = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(out3)
# out4 = layers.Flatten()(out3)
# out4 = layers.Dropout(0.4)(out4, training=True)
# out4 = layers.Dense(256, activation='relu')(out4)
# out5 = layers.Dropout(0.4)(out4,training=True)
# out5 = layers.Dense(128, activation='relu')(out5)
# model = Model(inputs=input_layer, outputs=out5)
# model.summary()


# Mayank's Architecture 2
```

```
# input_layer = layers.Input(shape = (im_height, im_width, im_depth, im_channel))
# out1 = layers.Conv3D(filters=8, kernel_size=(3,3,7), activation='relu',input_shape=(im_height, im_width, im_depth, im_channel),padding=
# out1 = DropBlock3D(0.7,3)(out1)
# out2 = layers.Conv3D(filters=16, kernel_size=(3,3,5), activation='relu',padding='same')(out1)
# out2 = DropBlock3D(0.7,3)(out2)
# out3 = layers.Conv3D(filters=32, kernel_size=(3,3,3), activation= 'relu')(out2)

# out3 = layers.Reshape((out3.shape[1], out3.shape[2], out3.shape[3]*out3.shape[4]))(out3)
# out3 = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(out3)

# # Add an attention layer here
# query = layers.Dense(64)(out3)                      # query vector for each pixel
# key = layers.Dense(64)(out3)                        # key vector for each pixel
# value = layers.Dense(64)(out3)                      # value vector for each pixel
# attention = layers.Attention()([query, key, value]) # compute dot-product attention
# attention = layers.Add()([attention, out3])         # add residual connection
# attention = layers.LayerNormalization()(attention)  # normalize the output

# out4 = layers.Flatten()(attention)
# out4 = layers.Dropout(0.4)(out4, training=True)
# out4 = layers.Dense(256, activation='relu')(out4)
# out5 = layers.Dropout(0.4)(out4,training=True)
# out5 = layers.Dense(128, activation='relu')(out5)
# model = Model(inputs=input_layer,outputs=out5)
# model.summary()



# import tensorflow as tf
# print("Tensorflow version " + tf.__version__)

# try:
#   tpu = tf.distribute.cluster_resolver.TPUClusterResolver()  # TPU detection
#   print('Running on TPU ', tpu.cluster_spec().as_dict()['worker'])
# except ValueError:
#   raise SystemExit('Not connected to a TPU runtime; please see the previous cell in this notebook for instructions!')

# tf.config.experimental_connect_to_cluster(tpu)
# tf.tpu.experimental.initialize_tpu_system(tpu)
# tpu_strategy = tf.distribute.TPUStrategy(tpu)


import tensorflow as tf

# Check if a GPU is available
if tf.test.gpu_device_name():
    print('Default GPU Device: {}'.format(tf.test.gpu_device_name()))
else:
    print("Please install GPU version of TF")

# Specify the device to use for computations
    # Your code here

     Default GPU Device: /device:GPU:0


# Mayank's Architecture 3
with tf.device('/device:GPU:0'):

  input_layer = layers.Input(shape = (im_height, im_width, im_depth, im_channel))
  out1 = layers.Conv3D(filters=8, kernel_size=(3,3,7), activation='relu',input_shape=(im_height, im_width, im_depth, im_channel),padding=
  out1 = DropBlock3D(0.7,3)(out1)
  out2 = layers.Conv3D(filters=16, kernel_size=(3,3,5), activation='relu',padding='same')(out1)
  out2 = DropBlock3D(0.7,3)(out2)

  out3_conv3d = layers.Conv3D(filters=32, kernel_size=(3,3,3), activation= 'relu')(out2)


  reshape_conv3d = layers.Reshape((out3_conv3d.shape[1], out3_conv3d.shape[2], out3_conv3d.shape[3]*out3_conv3d.shape[4]))(out3_conv3d)
  out3_conv2d = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(reshape_conv3d)
  SA_conv2d = layers.Attention()([out3_conv2d, out3_conv2d])
  Concat_conv2d = layers.Attention(score_mode="concat")([SA_conv2d, out3_conv2d])

  SA_conv3d = layers.Attention()([out3_conv3d,out3_conv3d])
  Concat_conv3d = layers.Attention(score_mode="concat")([SA_conv3d, out3_conv3d ])
  reshape_SA_conv3d = layers.Reshape((Concat_conv3d.shape[1], Concat_conv3d.shape[2], Concat_conv3d.shape[3]*Concat_conv3d.shape[4]))(Cor
  conv2d = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(reshape_SA_conv3d)

  cross_attention = layers.Attention()([conv2d, Concat_conv2d])
  ele_wise_summation = layers.Add()([cross_attention, out3_conv2d])
  normalization = layers.LayerNormalization()(ele_wise_summation) # normalize the output
```

Saved successfully!

```
out4 = layers.Flatten()(normalization)
out4 = layers.Dropout(0.4)(out4, training=True)
out4 = layers.Dense(256, activation='relu')(out4)
out5 = layers.Dropout(0.4)(out4,training=True)
out5 = layers.Dense(128, activation='relu')(out5)
model = Model(inputs=input_layer, outputs=out5)
model.summary()
```

```
 drop_block3d (DropBlock3D)     (None, 11, 11, 30,   0           ['conv3d[0][0]']
                                8)

 conv3d_1 (Conv3D)              (None, 11, 11, 30,   5776        ['drop_block3d[0][0]']
                                16)

 drop_block3d_1 (DropBlock3D)   (None, 11, 11, 30,   0           ['conv3d_1[0][0]']
                                16)

 conv3d_2 (Conv3D)              (None, 9, 9, 28, 32  13856       ['drop_block3d_1[0][0]']
                                )

 attention_2 (Attention)        (None, 9, 9, 28, 32  0           ['conv3d_2[0][0]',
                                )                                  'conv3d_2[0][0]']

 reshape (Reshape)              (None, 9, 9, 896)    0           ['conv3d_2[0][0]']

 attention_3 (Attention)        (None, 9, 9, 28, 32  1           ['attention_2[0][0]',
                                )                                  'conv3d_2[0][0]']

 conv2d (Conv2D)                (None, 7, 7, 64)     516160      ['reshape[0][0]']

 reshape_1 (Reshape)            (None, 9, 9, 896)    0           ['attention_3[0][0]']

 attention (Attention)          (None, 7, 7, 64)     0           ['conv2d[0][0]',
                                                                  'conv2d[0][0]']

 conv2d_1 (Conv2D)              (None, 7, 7, 64)     516160      ['reshape_1[0][0]']

                                (None, 7, 7, 64)     1           ['attention[0][0]',
                                                                  'conv2d[0][0]']

 attention_4 (Attention)        (None, 7, 7, 64)     0           ['conv2d_1[0][0]',
                                                                  'attention_1[0][0]']

 add (Add)                      (None, 7, 7, 64)     0           ['attention_4[0][0]',
                                                                  'conv2d[0][0]']

 layer_normalization (LayerNorm  (None, 7, 7, 64)    128         ['add[0][0]']
 alization)

 flatten (Flatten)              (None, 3136)         0           ['layer_normalization[0][0]']

 dropout (Dropout)              (None, 3136)         0           ['flatten[0][0]']

 dense (Dense)                  (None, 256)          803072      ['dropout[0][0]']

 dropout_1 (Dropout)            (None, 256)          0           ['dense[0][0]']

 dense_1 (Dense)                (None, 128)          32896       ['dropout_1[0][0]']

==================================================================================================
Total params: 1,888,562
Trainable params: 1,888,562
Non-trainable params: 0
__________________________________________________________________________________________________
```

Saved successfully!

```
# with tpu_strategy.scope(): # creating the model in the TPUStrategy scope means we will train the model on the TPU
#   model = Model(inputs=input_layer, outputs=out5)
# model.summary()
```

```
with tf.device('/device:GPU:0'):

  class Prototypical(Model):
      def __init__(self, model, w, h, d, c):
          super(Prototypical, self).__init__()
          self.w, self.h, self.d, self.c = w, h, d, c
          self.encoder = model

      def call(self, support, query, support_labels, query_labels, K, C, N,n_times,training=True):
        n_class = C                                                          #10
        n_support = K                                                        #5
        n_query = N                                                          #15
        temperature = 1.8

        if training == True :
```

```
        loss = 0
        mc_predictions = []                                                 # list of predictions for multiple passes
        for i in range(n_times) :
          y = np.zeros((int(C*N),C))                                        #(150,10)
          for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                       # creation of 1-hot for true labels
            y[i][x] = 1.                                                    # n_times passing every query image for calculating var
          cat = tf.concat([support,query], axis=0)                          # [200,9,9,20,1])
          z = self.encoder(cat)                                             # [200, 320]
          # Divide embedding into support and query
          z_prototypes = tf.reshape(z[:n_class * n_support],[n_class, n_support, z.shape[-1]])   #[10, 5, 320])
          # Prototypes are means of n_support examples
          z_prototypes = tf.math.reduce_mean(z_prototypes, axis=1)          #[10, 320]
          z_query = z[n_class * n_support:]                                 #[150, 320]
          # Calculate distances between query and prototypes
          dists = calc_euclidian_dists(z_query, z_prototypes)               #[150, 10]
          # log softmax of calculated distances
          log_p_y = tf.nn.log_softmax(-dists, axis=-1)                      #[150, 10]
          loss1 = -tf.reduce_mean((tf.reduce_sum(tf.multiply(y, log_p_y), axis=-1)))   #loss for the current pass
          loss += loss1                                                     # adding loss for each pass
          softmax_predictions = tf.nn.softmax(-dists, axis=-1)              # prediction probability for the search-space classes p
          new_distance = dists/temperature                                  # T = 1.1 to 3.0 for Temperature Scaling
          predictions = tf.nn.softmax(-new_distance, axis=-1)               # prediction probability with temperature scaling
          # print('Loop Number :', i)
          # print("A" * 10)
          # print("Dists : ", dists[0])
          # print("B" * 10)
          # print('New Distance (T applied): ', new_distance[0])
          # print("C" * 10)
          # print("Softmax Predictions : ", softmax_predictions[0])
          # print("D" * 10)
          # print("Temperature Scaling Predictions : ", predictions[0])
          # print()
          mc_predictions.append(predictions)
```

Saved successfully!

```
            ex(query_labels[i])                                             # creation of 1-hot for true labels
            y[i][x] = 1.
        mc_predictions = tf.convert_to_tensor(np.reshape(np.asarray(mc_predictions),(n_times,int(C*N),C)))  #(n_times,150,10)
        std_predictions = tf.math.reduce_std(mc_predictions,axis=0)
        std = tf.reduce_sum(tf.reduce_sum(tf.multiply(std_predictions,y),axis=1))
        # print('std', std)
        loss += mc_loss_weight*std
        temp_list.append(std)
        # calculating mean accuracy
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)            # mean prediction probability for each class (150,10)
        mean_eq = tf.cast(tf.equal(                                         # accuracy for the current pass
            tf.cast(tf.argmax(mean_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(y,axis=-1), tf.int32)), tf.float32)
        mean_accuracy = tf.reduce_mean(mean_eq)
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)            # mean prediction probability for each class (150,10)
        return loss, mean_accuracy, mean_predictions
      if training == False :
        loss = 0
        mc_predictions = []                                                 # list of predictions for multiple passes
        for i in range(n_times) :                                           # n_times passing the query images for variance calcula
          y = np.zeros((int(C*N),C))                                        # (150,10)
          for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                       # creation of 1-hot for the true labels
            y[i][x] = 1.
          # merge support and query to forward through encoder
          cat = tf.concat([support,query], axis=0)                          # [200,9,9,20,1]
          z = self.encoder(cat)                                             # [200, 320]
          # Divide embedding into support and query
          z_prototypes = tf.reshape(z[:n_class * n_support],[n_class, n_support, z.shape[-1]])   #[10, 5, 320])
          # Prototypes are means of n_support examples
          z_prototypes = tf.math.reduce_mean(z_prototypes, axis=1)          #[10, 320]
          z_query = z[n_class * n_support:]                                 #[150, 320]
          # Calculate distances between query and prototypes
          dists = calc_euclidian_dists(z_query, z_prototypes)               #[150, 10]
          # log softmax of calculated distances
          log_p_y = tf.nn.log_softmax(-dists, axis=-1)                      #[150, 10]
          loss1 = -tf.reduce_mean((tf.reduce_sum(tf.multiply(y, log_p_y), axis=-1)))
          loss += loss1
          softmax_predictions = tf.nn.softmax(-dists, axis=-1)              # prediction probability for the search-space classes p
          new_distance = dists/temperature                                  # T = 1.1 to 3.0 for Temperature Scaling
          predictions = tf.nn.softmax(-new_distance, axis=-1)               # prediction probability with temperature scaling
          # print('Loop Number : ', i)
          # print("A" * 10)
          # print("Dists : ", dists[0])
          # print("B" * 10)
          # print('New Distance (T applied): ', new_distance[0])
```

```
            # print("C" * 10)
            # print("Softmax Predictions : ", softmax_predictions[0])
            # print("D" * 10)
            # print("Temperature Scaling Predictions : ", predictions[0])
            mc_predictions.append(predictions)
        y = np.zeros((int(C*N),C))                                          # (150,10)
        for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                         # creation of 1-hot for the true labels
            y[i][x] = 1.
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)              # mean prediction probability for each class (150,10)
        mean_eq = tf.cast(tf.equal(                                           # accuracy for the current pass
            tf.cast(tf.argmax(mean_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(y,axis=-1), tf.int32)), tf.float32)
        mean_accuracy = tf.reduce_mean(mean_eq)
        mean_pred_index = tf.argmax(mean_predictions,axis=1)
        # mean class-wise accuracies
        mean_correct_class = [[] for i in range(tC)]
        mean_correct_pred = [[] for i in range(tC)]
        classwise_mean_acc = [[] for i in range(tC)]
        for i in range(int(C*N)):
          x = support_labels.index(query_labels[i])
          mean_correct_class[x].append('4')
          if(mean_pred_index[i] == x) :
            mean_correct_pred[x].append('4')
        for i in range(tC) :
          z = len(mean_correct_pred[i])/len(mean_correct_class[i])
          classwise_mean_acc[i].append(z)
        #std calculation
        std = 0
        for i in range(int(C*N)) :
          x = support_labels.index(query_labels[i])
          p_i = np.array([p[i,:] for p in mc_predictions])
          std_i = tf.math.reduce_std(p_i,axis=0)
          std_i_true = std_i[x]
                                                                            # adding std of each class
```

Saved successfully!

```
        y = np.zeros((int(C*N),C))                                          # (150,10)
        for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                         # creation of 1-hot for the true labels
            y[i][x] = 1.
        return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y


      def save(self, model_path):
        self.encoder.save(model_path)

      def load(self, model_path):
        self.encoder(tf.zeros([1, self.w, self.h, self.c]))
        self.encoder.load_weights(model_path)
```

Data Loading

```
with tf.device('/device:GPU:0'):

  def loadData(name):
      data = sio.loadmat('/content/drive/My Drive/Data/indian_pines_corrected.mat')['indian_pines_corrected']
      labels = sio.loadmat('/content/drive/My Drive/Data/indian_pines_gt.mat')['indian_pines_gt']
      return data, labels
  # without reduction of 200 channels to 30 channels, memory error while creating cube
  def applyPCA(X, numComponents):
      newX = np.reshape(X, (-1, X.shape[2]))
      pca = PCA(n_components=numComponents, whiten=True)
      newX = pca.fit_transform(newX)
      newX = np.reshape(newX, (X.shape[0],X.shape[1], numComponents))
      return newX, pca

  def padWithZeros(X, margin):
      newX = np.zeros((X.shape[0] + 2 * margin, X.shape[1] + 2* margin, X.shape[2]))
      x_offset = margin
      y_offset = margin
      newX[x_offset:X.shape[0] + x_offset, y_offset:X.shape[1] + y_offset, :] = X
      return newX

  def createImageCubes(X, y, windowSize, removeZeroLabels = True):
      margin = int((windowSize - 1) / 2)
      zeroPaddedX = padWithZeros(X, margin=margin)  # X :(145, 145, 30) --> (195, 195, 30) with window =25
      # split patches
      patchesData = np.zeros((X.shape[0] * X.shape[1], windowSize, windowSize, X.shape[2]))  # (21025, 25, 25, 30)
      patchesLabels = np.zeros((X.shape[0] * X.shape[1]))  # (21025,)
```

```
    patchIndex = 0

    for r in range(margin, zeroPaddedX.shape[0] - margin):
        for c in range(margin, zeroPaddedX.shape[1] - margin):
            patch = zeroPaddedX[r - margin:r + margin + 1, c - margin:c + margin + 1]
            patchesData[patchIndex, :, :, :] = patch
            patchesLabels[patchIndex] = y[r-margin, c-margin]
            patchIndex = patchIndex + 1

    patchesData = np.expand_dims(patchesData, axis=-1)
    return patchesData,patchesLabels
```

```
dataset1 = 'indian_pines'                                        # 16 classes
ip_x1, ip_y = loadData(dataset1)                              #((512, 217, 204), (512, 217))
ip_x2,pca = applyPCA(ip_x1,numComponents=30)                   # ((512, 217, 20), (512, 217))
ip_X,ip_Y = createImageCubes(ip_x2, ip_y, windowSize=11)   #(111104, 9, 9, 20, 1) (111104,)
print(ip_X.shape,ip_Y.shape)
```

```
(21025, 11, 11, 30, 1) (21025,)
```

```
def patches_class(X,Y,n) :
  n_classes = n
  patches_list = []
  for i in range(1,n_classes+1):   # not considering class 0
    patchesData_Ith_Label = X[Y==i,:,:,:,:]
    patches_list.append(patchesData_Ith_Label)
  return patches_list
```

```
patches_class_ip = patches_class(ip_X,ip_Y,16) # class_wise list of patches #(16,) for class 0: (2009, 9, 9, 20, 1)
```

```
train_class_indices = [1,2,4,5,7,9,10,11,13,14]
test_class_indices = [0,3,6,8,12,15]
                                    _ip[i] for i in train_class_indices]        #(10)
                                    ip[i] for i in test_class_indices]        #(6)
                                    1,12,14,15]
test_class_labels = [1,4,7,9,13,16]     #[11...16]
```

Saved successfully!

Prepare Training and validation Dataset

Create DataLoader

```
C = 5  # n_class
K1 = 5   # n_support
N = 15   # n_query
tC = 3   # classes in a test episode
im_height,im_width,im_depth = 11,11,30
```

```
with tf.device('/device:GPU:0'):

  def new_episode(patches_list,K,C,N,class_labels) :
    selected_classes = np.random.choice(class_labels,C,replace=False)  # Randomly choice 5 Classes out of classes available
    tsupport_patches = []
    tquery_patches = []
    query_labels = []
    support_labels = list(selected_classes)
    for x in selected_classes :
      sran_indices = np.random.choice(len(patches_list[x-1]),K,replace=False)  # for class no X-1: select random sample no
      support_patches = patches_list[x-1][sran_indices,:,:,:,:]
      qran_indices = np.random.choice(len(patches_list[x-1]),N,replace=False)  # N Samples for Query
      query_patches = patches_list[x-1][qran_indices,:,:,:,:]
    # Support and Query patches belong to same Class
      for i in range(N) :
        query_labels.append(x)    # N Samples for Query
      tquery_patches.extend(query_patches)
      tsupport_patches.extend(support_patches)
    temp1 = list(zip(tquery_patches, query_labels))
    random.shuffle(temp1)        # By Doing Shuffling, Support, Query Same class combination got mismatched - mitigated by support index
    tquery_patches, query_labels = zip(*temp1)
    tquery_patches = tf.convert_to_tensor(np.reshape(np.asarray(tquery_patches),(C*N,im_height,im_width,im_depth,1)),dtype=tf.float32)
    tsupport_patches = tf.convert_to_tensor(np.reshape(np.asarray(tsupport_patches),(C*K,im_height,im_width,im_depth,1)),dtype=tf.float32
    return tquery_patches, tsupport_patches, query_labels, support_labels
```

```
with tf.device('/device:GPU:0'):
  tquery_patches, tsupport_patches, query_labels, support_labels = new_episode(patches_class_ip,K1,C,N,train_class_labels)
```

Training

```
with tf.device('/device:GPU:0'):
  ProtoModel = Prototypical(model,im_width, im_height, im_depth, im_channel)
  optimizer = tf.keras.optimizers.Adam(0.00001)
  n_times = 5

  #std_list = []
```

```
with tf.device('/device:GPU:0'):

  # Metrics to gather
  train_loss = tf.metrics.Mean(name='train_loss')
  train_acc = tf.metrics.Mean(name='train_accuracy')

  def train_step(support, query, support_labels, query_labels, K, C, N):
      # Forward & update gradients
      with tf.GradientTape() as tape:
          loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N, n_times,training=True
      gradients = tape.gradient(loss, model.trainable_variables)
      optimizer.apply_gradients(zip(gradients, model.trainable_variables))

      # Log loss and accuracy for step
      train_loss(loss)
      train_acc(mean_accuracy)
```

```
checkpoint_dir = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IP/
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

Saved successfully!

```
  n_episodes = 100
  temp_list = []
  for epoch in range(50): # n_epochs-140
      train_loss.reset_states()
      train_acc.reset_states()

      for epi in range(n_episodes):
          tquery_patches, tsupport_patches, query_labels, support_labels = new_episode(patches_class_ip,K1,C,N,train_class_labels)
          train_step(tsupport_patches, tquery_patches,support_labels, query_labels, K1, C, N)
          template = 'Epoch {}, Episode {}, Train Loss: {:.2f}, Train Accuracy: {:.2f}'
          print(template.format(epoch+1, epi+1,train_loss.result(),train_acc.result()*100))
      if epoch % 5 == 0 and epoch != 0 :
          checkpoint.save(file_prefix = checkpoint_prefix)
```

```
Epoch 50, Episode 77, Train Loss: 10.57, Train Accuracy: 99.77
Epoch 50, Episode 78, Train Loss: 10.42, Train Accuracy: 99.78
Epoch 50, Episode 79, Train Loss: 10.54, Train Accuracy: 99.78
Epoch 50, Episode 80, Train Loss: 10.54, Train Accuracy: 99.78
Epoch 50, Episode 81, Train Loss: 10.54, Train Accuracy: 99.79
Epoch 50, Episode 82, Train Loss: 10.55, Train Accuracy: 99.79
Epoch 50, Episode 83, Train Loss: 10.51, Train Accuracy: 99.79
Epoch 50, Episode 84, Train Loss: 10.55, Train Accuracy: 99.79
Epoch 50, Episode 85, Train Loss: 10.49, Train Accuracy: 99.80
Epoch 50, Episode 86, Train Loss: 10.63, Train Accuracy: 99.78
Epoch 50, Episode 87, Train Loss: 10.56, Train Accuracy: 99.79
Epoch 50, Episode 88, Train Loss: 10.57, Train Accuracy: 99.79
Epoch 50, Episode 89, Train Loss: 10.62, Train Accuracy: 99.78
Epoch 50, Episode 90, Train Loss: 10.64, Train Accuracy: 99.78
Epoch 50, Episode 91, Train Loss: 10.67, Train Accuracy: 99.77
Epoch 50, Episode 92, Train Loss: 10.72, Train Accuracy: 99.77
Epoch 50, Episode 93, Train Loss: 10.74, Train Accuracy: 99.77
Epoch 50, Episode 94, Train Loss: 10.69, Train Accuracy: 99.77
Epoch 50, Episode 95, Train Loss: 10.69, Train Accuracy: 99.76
Epoch 50, Episode 96, Train Loss: 10.64, Train Accuracy: 99.76
Epoch 50, Episode 97, Train Loss: 10.67, Train Accuracy: 99.77
Epoch 50, Episode 98, Train Loss: 10.67, Train Accuracy: 99.77
Epoch 50, Episode 99, Train Loss: 10.63, Train Accuracy: 99.77
```

Tuning_5

```
with tf.device('/device:GPU:0'):

  tune_set_5 = [[] for i in range(6)]
  for j in range(6) :
    tune_set_5[j] = test_patches_class[j][:5,:,:,:,:]   # for each class first 5 samples taken
  std_tune_5 = []
```

```
with tf.device('/device:GPU:0'):
```

Saved successfully!

```
                                  ,test_class_labels) :
                                  ice(test_class_labels,tC,replace=False)
    support_labels  = list(selected_classes)
    query_labels = []
    support_patches = []
    query_patches = []
    for x in selected_classes :
      y = test_class_labels.index(x)
      np.random.shuffle(tune_set[y])
      support_imgs = tune_set[y][:tK,:,:,:,:]    #Support 1, Query 4
      query_imgs = tune_set[y][tK:5,:,:,:,:]
      support_patches.extend(support_imgs)
      query_patches.extend(query_imgs)
      for i in range(tN) :
        query_labels.append(x)
    temp1 = list(zip(query_patches, query_labels))
    random.shuffle(temp1)
    query_patches, query_labels = zip(*temp1)
    query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(tC*tN,im_height,im_width,im_depth,1)),dtype=tf.float32)
    support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(tC*tK,im_height,im_width,im_depth,1)),dtype=tf.float32

    return query_patches, support_patches, query_labels, support_labels
```

```
with tf.device('/device:GPU:0'):

  query_patches, support_patches, query_labels, support_labels = tune_episode(tune_set_5,4,1,4,test_class_labels)
```

```
with tf.device('/device:GPU:0'):

  # Metrics to gather
  tune_loss = tf.metrics.Mean(name='tune_loss')
  tune_acc = tf.metrics.Mean(name='tune_accuracy')

  def tune_step(support, query, support_labels, query_labels, K, C, N):
      # Forward & update gradients
      with tf.GradientTape() as tape:
          loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
      gradients = tape.gradient(loss, model.trainable_variables)
      optimizer.apply_gradients(zip(gradients, model.trainable_variables))

      # Log loss and accuracy for step
      tune_loss(loss)
      tune_acc(mean_accuracy)
```

```
with tf.device('/device:GPU:0'):
  checkpoint_dir1 = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/
  checkpoint_prefix1 = os.path.join(checkpoint_dir1, "ckpt")
  checkpoint = tf.train.Checkpoint(optimizer= optimizer,
                                   ProtoModel = ProtoModel)
```

```
with tf.device('/device:GPU:0'):
  n_episodes = 100
  temp_list = []
  std_tune_5 = []
  for epoch in range(41):
      tune_loss.reset_states()
      tune_acc.reset_states()
      for epi in range(n_episodes+1):
          tquery_patches, tsupport_patches, query_labels, support_labels = tune_episode(tune_set_5,3,1,4,test_class_labels)
          tune_step(tsupport_patches, tquery_patches,support_labels, query_labels, 1, 3, 4)

      template = 'Epoch {}, Tune Loss: {:.2f}, Tune Accuracy: {:.2f}'
      print(template.format(epoch+1,tune_loss.result(),tune_acc.result()*100))
      if (epoch+1)%5 == 0 :
        checkpoint.save(file_prefix = checkpoint_prefix1)
```

```
Epoch 1, Tune Loss: 2.62, Tune Accuracy: 99.75
Epoch 2, Tune Loss: 0.90, Tune Accuracy: 100.00
Epoch 3, Tune Loss: 0.54, Tune Accuracy: 100.00
Epoch 4, Tune Loss: 0.45, Tune Accuracy: 100.00
Epoch 5, Tune Loss: 0.32, Tune Accuracy: 100.00
Epoch 6, Tune Loss: 0.32, Tune Accuracy: 100.00
Epoch 7, Tune Loss: 0.19, Tune Accuracy: 100.00
Epoch 8, Tune Loss: 0.29, Tune Accuracy: 100.00
Epoch 9, Tune Loss: 0.24, Tune Accuracy: 100.00
Epoch 10, Tune Loss: 0.16, Tune Accuracy: 100.00
Epoch 11, Tune Loss: 0.18, Tune Accuracy: 100.00
Epoch 12, Tune Loss: 0.13, Tune Accuracy: 100.00
Epoch 13, Tune Loss: 0.15, Tune Accuracy: 100.00
Accuracy: 100.00
Accuracy: 100.00
Epoch 16, Tune Loss: 0.13, Tune Accuracy: 100.00
Epoch 17, Tune Loss: 0.12, Tune Accuracy: 100.00
Epoch 18, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 19, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 20, Tune Loss: 0.10, Tune Accuracy: 100.00
Epoch 21, Tune Loss: 0.07, Tune Accuracy: 100.00
Epoch 22, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 23, Tune Loss: 0.11, Tune Accuracy: 100.00
Epoch 24, Tune Loss: 0.10, Tune Accuracy: 100.00
Epoch 25, Tune Loss: 0.10, Tune Accuracy: 100.00
Epoch 26, Tune Loss: 0.09, Tune Accuracy: 100.00
Epoch 27, Tune Loss: 0.14, Tune Accuracy: 100.00
Epoch 28, Tune Loss: 0.10, Tune Accuracy: 100.00
Epoch 29, Tune Loss: 0.07, Tune Accuracy: 100.00
Epoch 30, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 31, Tune Loss: 0.06, Tune Accuracy: 100.00
Epoch 32, Tune Loss: 0.07, Tune Accuracy: 100.00
Epoch 33, Tune Loss: 0.05, Tune Accuracy: 100.00
Epoch 34, Tune Loss: 0.06, Tune Accuracy: 100.00
Epoch 35, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 36, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 37, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 38, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 39, Tune Loss: 0.05, Tune Accuracy: 100.00
Epoch 40, Tune Loss: 0.07, Tune Accuracy: 100.00
Epoch 41, Tune Loss: 0.06, Tune Accuracy: 100.00
```

Saved successfully!

```
with tf.device('/device:GPU:0'):
  def test_episode(test_patches_class,test_class_labels,test_C,test_K,i,f) :
    selected_classes = test_class_labels[i:f]   # [1, 2, 3, 4, 5, 6, 7, 8]
    support_labels = list(selected_classes)
    query_labels = []
    support_patches = []
    query_patches = []
    for x in selected_classes :
      y = test_class_labels.index(x)
      support_imgs = test_patches_class[y][:test_K,:,:,:,:]
      query_imgs = test_patches_class[y][test_K:,:,:,:,:]
      support_patches.extend(support_imgs)
      query_patches.extend(query_imgs)
      for i in range(query_imgs.shape[0]) :
        query_labels.append(x)
    temp1 = list(zip(query_patches, query_labels))
    random.shuffle(temp1)
    query_patches, query_labels = zip(*temp1)
    x = len(query_labels)
    query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(x,im_height,im_width,im_depth,1)),dtype=tf.float32)
    support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(test_C*test_K,im_height,im_width,im_depth,1)),dtype=tf
```

```
    return query_patches, support_patches, query_labels, support_labels,x


with tf.device('/device:GPU:0'):

  query_patches, support_patches, query_labels, support_labels,x = test_episode(test_patches_class,test_class_labels,3,5,0,3)


with tf.device('/device:GPU:0'):
  # Metrics to gather
  test_loss = tf.metrics.Mean(name='test_loss')
  test_acc = tf.metrics.Mean(name='test_accuracy')
  def test_step(support, query, support_labels, query_labels, K, C, y):
      loss, mc_predictions, mean_accuracy, classwise_mean_acc, y = ProtoModel(support, query, support_labels, query_labels, K, C, y,n_tim
      return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y



for epoch in range(1): #1000
    test_loss.reset_states()
    test_acc.reset_states()

    # Class-wise Accuracy
    tquery_patches1, tsupport_patches1, query_labels1, support_labels1, x1 = test_episode(test_patches_class,test_class_labels,3,5,0,3)
    loss1, mc_predictions1, mean_accuracy1, classwise_mean_acc1, y1 = test_step(tsupport_patches1, tquery_patches1,support_labels1, query
    for i in range(tC) :
      print('class',i+1,classwise_mean_acc1[i])
    print('OA1',mean_accuracy1)
    print('loss',loss1)

    # Class-wise Accuracy
    tquery_patches2, tsupport_patches2, query_labels2, support_labels2, x2 = test_episode(test_patches_class,test_class_labels,3,5,3,6)
    loss2, mc_predictions2, mean_accuracy2, classwise_mean_acc2, y2 = test_step(tsupport_patches2, tquery_patches2,support_labels2, query
    for i in range(tC) :
      print('class',i+4,classwise_mean_acc2[i])
```

Saved successfully!

```
class 1 [1.0]
class 2 [1.0]
class 3 [1.0]
OA1 tf.Tensor(1.0, shape=(), dtype=float32)
loss tf.Tensor(16.838066, shape=(), dtype=float32)
class 4 [1.0]
class 5 [0.97]
class 6 [1.0]
OA2 tf.Tensor(0.980198, shape=(), dtype=float32)
loss tf.Tensor(39.1, shape=(), dtype=float32)
```

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
cm_pred1 = tf.argmax(mean_predictions1, axis=-1)
cm_pred2 = tf.argmax(mean_predictions2, axis=-1) + 3
cm_true1 = tf.argmax(y1,axis=-1)
cm_true2 = tf.argmax(y2,axis=-1) + 3
```

Overall Accuracy

```
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
overall_predictions = tf.concat([mean_predictions1,mean_predictions2],axis=0)
overall_true_labels = tf.concat([y1,y2],axis=0)
correct_pred = tf.cast(tf.equal(                                          # accuracy for the current pass
            tf.cast(tf.argmax(overall_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(overall_true_labels,axis=-1), tf.int32)), tf.float32)
o_acc = tf.reduce_mean(correct_pred)
print('Overall Accuracy : {:.2f}%'.format(o_acc * 100))
```

```
Overall Accuracy : 99.00%
```

Average Accuracy

```
from sklearn.metrics import accuracy_score
overall_prediction = tf.concat([cm_pred1,cm_pred2],axis=0)
overall_true_label = tf.concat([cm_true1,cm_true2],axis=0)
accuracy = accuracy_score(overall_true_label, overall_prediction)
print('Average Accuracy : {:.2f}%'.format(accuracy * 100))
```

```
Average Accuracy : 99.00%
```

Kappa Accuracy

```
import sklearn
kappa_acc = sklearn.metrics.cohen_kappa_score(overall_true_label, overall_prediction)
print('Kappa Accuracy : {:.2f}%'.format(kappa_acc * 100))
```

```
Kappa Accuracy : 98.60%
```

Confusion Matrix

```
results = confusion_matrix(overall_true_label,overall_prediction)
print ('Confusion Matrix :')
print(results)
print ('Report : ')
print (classification_report(overall_true_labels, overall_predictions))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-84-7595ae57c47d> in <cell line: 1>()
----> 1 results = confusion_matrix(overall_true_labels,overall_predictions)
      2 print ('Confusion Matrix :')
      3 print(results)
      4 print ('Report : ')
      5 print (classification_report(overall_true_labels, overall_predictions))

                              1 frames
/usr/local/lib/python3.9/dist-packages/sklearn/metrics/_classification.py in _check_targets(y_true,
y_pred)
     93
     96                 classification metrics can't handle a mix of {0} and {1} targets".format(
     97                     type_true, type_pred

ValueError: Classification metrics can't handle a mix of multilabel-indicator and continuous-
multioutput targets
```

Saved successfully!

0s completed at 7:27 PM